Web Application Vulnerability Scanner

Elevate Labs — Cyber Security Internship | 2025 | Nithish Nidumolu

1. Introduction

The Web Application Vulnerability Scanner is a Python-based security tool developed as part of the Elevate Labs Cyber Security Internship (2025). It is designed to automatically detect common web application vulnerabilities — specifically Cross-Site Scripting (XSS) and SQL Injection (SQLi) — both listed in the OWASP Top 10. The tool crawls HTML forms on a target URL, injects predefined attack payloads, and analyses server responses to identify vulnerable entry points. A Flask-based web interface hosted on Replit allows users to scan any target URL and view structured results directly in the browser.

2. Abstract

The scanner uses BeautifulSoup4 to parse and extract all HTML forms from a target web page. For each form, it injects XSS payloads (e.g., <script>alert('XSS')</script>) and checks whether the payload is reflected in the response to confirm Reflected XSS. It also injects SQLi payloads (e.g., ' OR 1=1--) and scans responses for database error strings (mysql, syntax, ORA-) to confirm SQL Injection. All findings are categorised by severity — HIGH for XSS, CRITICAL for SQLi — and saved to a structured report (scan_report.txt). The Flask interface renders colour-coded results with payload details and affected URLs.

3. Tools & Technologies

- Python 3.8+ — Core programming language
- Flask — Web framework for the UI and scan routing
- Requests — HTTP library for sending payloads to target URLs
- BeautifulSoup4 — HTML parser for form extraction and input field discovery
- Jinja2 — Dynamic rendering of scan results in HTML templates
- Replit — Cloud deployment platform

4. Steps Involved in Building the Project

- Project Setup: Created the folder structure with app.py, scanner.py, templates/ (index.html, results.html), and static/ (style.css). Installed Flask, Requests, and BeautifulSoup4.
- Form Crawling: Implemented get_forms(url) using Requests to fetch the page HTML and BeautifulSoup4 to extract all <form> elements. Built get_form_details() to extract action URL, HTTP method, and input field names.
- XSS Detection: Implemented scan_xss(url) to inject multiple XSS payloads into all discovered input fields. After each submission, the response HTML is checked for payload reflection to confirm Reflected XSS.
- SQL Injection Detection: Implemented scan_sqli(url) to inject SQLi payloads via GET and POST. Responses are scanned for database error keywords (mysql, syntax, ORA-, sqlite_error) to confirm vulnerability.
- Report Generation: Implemented generate_report(url, findings) to save all findings to scan_report.txt with target URL, timestamp, vulnerability type, severity, affected URL, and payload used.
- Flask Routes: Defined / (home) and /scan (POST) routes. The scan route calls the XSS and SQLi functions, collects findings, generates the report, and passes results to the Jinja2 template.
- Frontend: Built index.html with a URL input form and results.html with colour-coded severity badges — CRITICAL in red, HIGH in orange — rendered via Jinja2 templating.
- Deployment: Configured the Flask server for Replit and deployed publicly at https://web-scan--nithishnidumolu.replit.app.

5. Conclusion

The Web Application Vulnerability Scanner successfully demonstrates automated XSS and SQL Injection detection in a deployable Python security tool. The modular scanner architecture separates form crawling, payload injection, response analysis, and report generation into independent, testable layers.

6. Features & Functionality

Core Features	
XSS Detection	Automatically injects multiple Cross-Site Scripting payloads (e.g., <script>alert('XSS')</script>) into all discovered form inputs. Checks server responses for payload reflection to confirm Reflected XSS vulnerabilities. Severity: HIGH.
SQL Injection Detection	Injects SQLi payloads (e.g., ' OR 1=1--) into all form fields via GET and POST. Scans responses for database error strings such as 'mysql', 'syntax error', and 'ORA-' to confirm SQL Injection. Severity: CRITICAL.
Automatic Form Crawling	Uses BeautifulSoup4 to automatically discover and extract all HTML forms, input fields, and action URLs from the target page — supporting both GET and POST form methods for accurate payload delivery.
Colour-Coded Severity Badges	Scan results are rendered with colour-coded badges: CRITICAL findings displayed in red for SQL Injection and HIGH in orange for XSS, enabling developers to immediately prioritise fixes.
Auto-Generated Scan Report	Every scan automatically saves a structured scan_report.txt report containing target URL, scan timestamp, total issue count, vulnerability type, severity, affected URL, and the exact payload that triggered each finding.
Clean Scan Result	If no vulnerabilities are detected, the results page displays a green 'No vulnerabilities detected' success message with a Scan Another URL button, reassuring the user of a clean target.
Remediation Verification	After developers apply fixes, the scanner can be re-run on the same target. A clean second scan confirms that XSS output encoding and SQLi parameterised queries are correctly implemented.
Modular & Extensible	The separation between the Flask routing layer and scanner.py logic means new vulnerability checks (CSRF, open redirect, header injection) can be added by extending scanner.py without touching the web layer.

7. Scenario-Based Case Study

Scenario: Arjun, a junior web developer at a software company, has just finished building a customer login portal. Before going live, his security team asks him to run a vulnerability check using the Web Application Vulnerability Scanner to identify any OWASP Top 10 issues.

Step 1 — Accessing the Scanner: Arjun opens his browser and navigates to https://web-scan--nithishnidumolu.replit.app. He sees the clean scanner interface with a URL input field and Start Scan button. He enters the target URL: http://testphp.vulnweb.com.

Step 2 — Initiating the Scan: Arjun clicks Start Scan. The Flask backend receives the URL via a POST request. BeautifulSoup4 fetches the page and extracts 3 HTML forms — a login form, a search form, and a registration form — ready for payload injection.

Step 3 — XSS Detection Result: The scanner injects XSS payloads into all input fields. The search form reflects the payload back in the HTML response without sanitisation. The scanner flags this as a HIGH severity Reflected XSS vulnerability and records the affected URL and payload.

Step 4 — SQL Injection Detection Result: SQLi payload ' OR 1=1-- is injected into the login form. The server returns a response containing a MySQL error string — confirming the login form is vulnerable to SQL Injection.

Step 5 — Reviewing Results & Report: Arjun sees the scan results page with colour-coded severity badges. He opens scan_report.txt — which lists 4 total findings with full payload details — and shares it with the development team for remediation.